

Respuesta a la contrapropuesta: Defensa y refinamiento del Modelo de Plataforma para la reutilización de software

Gonzalo Enrique Gutiérrez Castillo
Área de Digitales – ACIDE
Arequipa, Perú
gonzalog@acideperu.org
gonzalogut.99@gmail.com

Resumen—Se presenta una respuesta formal a la contrapropuesta titulada “*Reutilización basada en plantillas y composición como alternativa a librerías y OOP*”. Se reconocen las críticas legítimas al diseño inicial, se corrigen imprecisiones factuales sobre la implementación existente, y se demuestra — mediante fuentes autoritativas y evidencia de la industria— que las premisas centrales de la contrapropuesta contienen malinterpretaciones de los principios YAGNI y KISS. Se concluye que ambas propuestas convergen en el mismo mecanismo de distribución (copiar y adaptar), y que la diferencia real es terminológica, no arquitectónica.

Index Terms—reutilización, arquitectura de software, YAGNI, KISS, Clean Architecture, plataforma, plantillas, ERP.

I. INTRODUCCIÓN

La contrapropuesta [1] presenta críticas al modelo de reutilización propuesto en [2], planteando un enfoque basado en plantillas y composición como alternativa. Este documento responde a dichas críticas con tres objetivos:

1. Reconocer las críticas legítimas y las fallas del primer informe.
2. Demostrar, con fuentes autoritativas, que varias premisas de la contrapropuesta contienen malinterpretaciones de principios fundamentales de ingeniería de software.
3. Evidenciar que ambas propuestas convergen en la misma solución, y que la discusión debiera enfocarse en los detalles de implementación, no en una falsa dicotomía.

II. RECONOCIMIENTO DE CRÍTICAS LEGÍTIMAS

Es importante reconocer, con honestidad intelectual, los puntos donde la contrapropuesta identifica problemas reales.

II-A. El diseño de *Persona/Estudiante* fue insuficiente

La crítica al caso concreto de *Persona.Surname* versus *Estudiante.ApellidoPaterno/ApellidoMaterno* es legítima. La abstracción de *Persona* no contempló adecuadamente la realidad del dominio educativo peruano, donde el manejo de dos apellidos separados es un requisito legal y administrativo.

Sin embargo, que una abstracción esté mal diseñada no invalida el concepto de abstracción. El problema fue de *diseño*, no de *principio*. La entidad unificada de *Persona/Contacto* es

un patrón consolidado en la industria del software empresarial, adoptado independientemente por los principales ERPs del mundo (ver Sección V).

La solución correcta era diseñar *Persona* con campos de nombre estructurados que soporten múltiples formatos culturales — como lo hace SAP en su tabla BUT000 con *NAME_FIRST*, *NAME_LAST* y *NAME_LAST2*— en lugar de usar un único campo *Surname*.

II-B. Falta de claridad operativa

El primer informe presentó el *qué* y el *por qué*, pero no detalló suficientemente el *cómo*: gobernanza, ownership del código, resolución de conflictos entre librería y negocio, y procesos de actualización. Esto es un vacío legítimo que este documento no pretende minimizar.

II-C. Composición sobre herencia

La recomendación de composición sobre herencia es técnicamente correcta y ampliamente aceptada [12]. No obstante, es necesario aclarar una imprecisión factual: **la implementación en Colver ya usaba composición**. Estudiante no hereda de *Persona*; hereda de *AggregateRoot* (su raíz de agregado en DDD) y mantiene una referencia a *Persona* mediante *PersonId* (clave foránea). La relación 1:1 entre *Estudiante* y *Persona* es una relación de composición, no de herencia de clases.

II-D. Sobre la iniciativa unilateral

El punto sobre falta de consenso es válido en su forma. Sin embargo, es necesario contextualizar: la acción fue tomada porque, a pesar de que el equipo se comprometía reiteradamente con una visión de reutilización, no se materializaban acciones concretas para avanzar hacia ese objetivo. Se mantenía el *status quo* proyecto tras proyecto. Ante la inacción colectiva, se optó por presentar una implementación concreta que sirviera como punto de partida para la discusión — no como imposición, sino como catalizador.

III. SOBRE YAGNI Y KISS: ALCANCE, USO CORRECTO Y MALINTERPRETACIONES

Es necesario aclarar una posición fundamental: **no nos oponemos a YAGNI ni a KISS**. Son principios valiosos y necesarios para el desarrollo de software. Sin embargo, no operan en aislamiento: deben aplicarse **en armonía** con DRY (*Don't Repeat Yourself*), SSOT (*Single Source of Truth*), los principios SOLID, el paradigma SPLE (*Software Product Line Engineering*) y SINE (*Simple Is Not Easy*) [24]. Este último principio es especialmente relevante: la simplicidad no es lo mismo que la facilidad. Construir una solución simple requiere diseño deliberado y esfuerzo; lo fácil es escribir código rápido sin estructura, pero eso no es simple, es deuda técnica. Ningún principio tiene supremacía absoluta sobre los demás; la ingeniería de software consiste precisamente en encontrar el equilibrio correcto según el contexto.

III-A. Dónde YAGNI y KISS sí aplican: el modelado de entidades

Es importante ser precisos. La contrapropuesta no critica la arquitectura (Clean Architecture, CQRS, DDD) — ambas partes estamos de acuerdo en esos patrones—. Lo que critica es el **modelado de entidades**: que se haya modelado `Branch` (múltiples sedes) cuando Colver tiene una sola, que `Persona` tenga un campo `Surname` genérico en lugar de apellidos específicos, y que se hayan diseñado entidades pensando en múltiples rubros durante el desarrollo de un proyecto específico.

En este nivel, YAGNI tiene legitimidad. Fowler define “*presumptive feature*” como “*any code that supports a feature that isn't yet being made available for use*” [3]. Si modeláramos entidades para un dominio que genuinamente desconocemos, eso sería especulación y YAGNI aplicaría con toda razón.

Aceptamos esta crítica parcialmente: durante el desarrollo de Colver, el foco debió estar en resolver Colver primero. Ahí YAGNI y KISS son guías correctas.

III-B. Dónde YAGNI y KISS no aplican: reutilización conocida

Sin embargo, la contrapropuesta extiende YAGNI más allá de su alcance legítimo cuando lo usa para rechazar toda inversión en reutilización. Jeffries, co-creador de XP y del principio YAGNI, define su alcance así:

*“Always implement things when you **actually** need them, never when you just **foresee** that you need them.”* [5]

La palabra clave es *actually* versus *foresee*. Cuando la consultora ya ejecutó múltiples proyectos con los mismos patrones (`Persona`, `Auth`, `Auditoría`), la necesidad ya no es especulativa: *actually need* ya se cumplió.

Fowler refuerza esta distinción:

“If you do something for a future need that doesn't actually increase the complexity of the software, then there's no reason to invoke yagni.” [3]

Modelar una entidad `Branch` con un FK no aumenta la complejidad del sistema de forma significativa, pero habilita la

reutilización para clientes con múltiples sedes. Es importante notar que respetar YAGNI aquí no significa eliminar la entidad: significa **no implementar lógica de cambio de sede, creación de nuevas sedes ni interfaces de gestión multi-sede** mientras el cliente no lo solicite. La entidad existe en el modelo de datos; los features asociados no, hasta que se necesiten.

Este no es un escenario hipotético. En un proyecto anterior de la consultora — el sistema para Clínica Juan Pablo II Dermatología Láser y Estética S.A.C. (RUC 20603130295)— se desarrolló inicialmente sin soporte multi-sede porque el cliente solicitó gestión global. Cuando posteriormente la clínica requirió operar por sedes, el backend no podía refactorizarse sin riesgo, y fue necesario aplicar parches inseguros que comprometían la integridad del sistema. Si la entidad `Branch` hubiera existido desde el modelo de datos inicial — sin lógica adicional, solo la estructura— la adaptación habría sido un feature nuevo en lugar de una cirugía de emergencia.

La propuesta inicial de librerías para Colver no surgió de una abstracción arbitraria ni de especulación teórica: fue construida sobre el conocimiento recogido en esta clase de antecedentes reales. Los patrones que se propuso estandarizar (`Persona`, `Organización`, `Sedes`, `Autenticación`) son exactamente los que generaron fricción y parches en proyectos anteriores cuando no estaban previstos en la estructura. Aplicar YAGNI a este tipo de decisiones estructurales es confundir simplicidad con recorte.

III-C. YAGNI sin calidad es “skimping”

Jeffries distingue explícitamente entre YAGNI legítimo y recorte de calidad:

“YAGNI, on its own, might not be safe... But when we are keeping the design quality, code quality, and test quality high, YAGNI is quite safe.” [4]

Su conclusión es categórica:

“YAGNI + clean code? Yes. Skimping? No.” [4]

Fowler complementa desde la perspectiva económica:

“Yagni is not a justification for neglecting the health of your code base. Yagni requires (and enables) malleable code.” [3]

III-D. Qué significa aplicar YAGNI y KISS correctamente en la consultora

Proponemos una interpretación razonable de ambos principios para el contexto específico de la consultora:

1. **YAGNI para features del cliente:** No modelar funcionalidades que el cliente actual no ha solicitado. Si el colegio no necesita facturación electrónica hoy, no se construye. Esto es YAGNI legítimo.
2. **KISS para la implementación:** Cada feature se implementa de la forma más simple que resuelva el problema. Sin capas innecesarias, sin abstracciones especulativas dentro del proyecto.
3. **DRY y SSOT entre proyectos:** Cuando un patrón ya se repitió en múltiples proyectos (`Persona`, `Auth`, `Auditoría`), se consolida en el repositorio base. Esto no viola YAGNI

porque la necesidad ya está demostrada; y no viola KISS porque reduce la complejidad global al eliminar duplicación.

4. **SINE para el repositorio base:** Construir plantillas reutilizables bien diseñadas *no es fácil*, pero el resultado *sí es simple*. La simplicidad del producto final justifica el esfuerzo de diseño [24].

Esta interpretación es coherente con las 4 reglas de diseño simple de Kent Beck, ordenadas por prioridad [9]:

1. Pasa los tests.
2. Revela intención.
3. **No tiene duplicación.**
4. Tiene el mínimo de elementos.

La regla 3 (no duplicación) tiene **mayor prioridad** que la regla 4 (mínimos elementos). Copiar y pegar el mismo código de autenticación, personas y auditoría en cada proyecto viola la regla 3, que es más importante que “tener menos archivos”. Bajo la definición original de Beck, el repositorio base **es más simple** que duplicar código entre proyectos.

III-E. El retorno de la inversión en diseño

Fowler, en su *Design Stamina Hypothesis*, demuestra que la inversión en diseño se recupera rápidamente:

“You can save short-term time by neglecting design, but this accumulates Technical Debt which will slow your productivity later.” [7]

“I take the view that [the design payoff line] is much lower than most people think: usually weeks not months.” [7]

Y sobre la calidad interna:

“High internal quality reduces the cost of future features, meaning that putting the time into writing good code actually reduces cost. [...] The ‘cost’ of high internal quality software is negative.” [8]

Robert C. Martin lo resume:

“If you think good architecture is expensive, try bad architecture.” [6]

IV. IMPRECISIONES FACTUALES EN LA CONTRAPROPUESTA

IV-A. Se critica un strawman: librería rígida vs. modelo propuesto

La contrapropuesta dedica la mayor parte de sus críticas al concepto de “librerías rígidas que no se pueden tocar”. Sin embargo, el informe original [2] explica explícitamente el modelo *shadcn*:

“Cada proyecto toma lo que necesita y lo moldea libremente. Las modificaciones en un proyecto no afectan a los demás.” [2]

La contrapropuesta propone exactamente lo mismo: plantillas que se copian, se pegan y se adaptan. **Es el mismo mecanismo de distribución con diferente terminología.** La Tabla I muestra la equivalencia.

Cuadro I
CONVERGENCIA ENTRE AMBAS PROPUESTAS

Propuesta original	Contrapropuesta
Repositorio base	Repo. de plantillas
Copiar y adaptar (<i>shadcn</i>)	Copiar y pegar
Clean Architecture + DDD	Clean Arch. + DDD
Proyecto dueño del código	Proyecto dueño del código
No modificar repo base por un cliente	Plantillas indep. de proyectos

IV-B. No es overengineering: los flujos eran conocidos

La contrapropuesta argumenta que se intentó “*replicar completamente toda la estructura utilizada por Odoo*” [1]. Esto es incorrecto. Las entidades implementadas en las librerías (Persona, Organización, Empleado, Autenticación) no son especulaciones teóricas: son patrones que el equipo ya había desarrollado en proyectos anteriores.

George Fairbanks proporciona el marco adecuado para evaluar cuándo la inversión arquitectónica está justificada:

“There is no need for meticulous designs when risks are small, nor any excuse for sloppy designs when risks threaten your success.” [10]

En el contexto de una consultora con múltiples proyectos, clientes repetidos y rotación de personal, el riesgo de no estandarizar es alto. La inversión arquitectónica está justificada.

IV-C. El ejemplo de “Colver solo tiene una sede” es falaz

Que el primer cliente tenga una única sede no invalida modelar la entidad *Branch*. Esta entidad añade un FK a la base de datos y permite que el sistema funcione para clientes con *N* sedes sin modificación alguna. El costo marginal es mínimo (una tabla, una clave foránea) y el beneficio de reutilización es alto. Fowler lo expresa directamente:

“If you do something for a future need that doesn’t actually increase the complexity of the software, then there’s no reason to invoke yagni.” [3]

V. LA ABSTRACCIÓN DE PERSONA: ESTÁNDAR DE LA INDUSTRIA

La contrapropuesta trata la entidad unificada de Persona como un ejemplo de sobre-ingeniería. Sin embargo, este patrón es el estándar de facto en todo el software empresarial del mundo. Cada uno de los principales ERPs lo ha adoptado de forma independiente.

V-A. Odoo: *res.partner*

Odoo utiliza un único modelo *res.partner* para **todos** los contactos del sistema: individuos, empresas, clientes, proveedores, empleados y direcciones. Los roles se asignan mediante campos de rango: *customer_rank* y *supplier_rank* [13]. Cada módulo del ERP (Ventas, Compras, Contabilidad, Inventario, CRM, RRHH) referencia *res.partner*. Una misma empresa puede ser simultáneamente cliente y proveedor.

V-B. SAP S/4HANA: Business Partner (BUT000)

SAP reemplazó sus tablas maestras separadas — KNA1 (cliente) y LFA1 (proveedor)— por un **Business Partner** unificado en BUT000. Los roles se asignan en una tabla separada BUT100, permitiendo N roles por entidad con fechas de validez [14]. Notablemente, BUT000 soporta explícitamente múltiples apellidos: NAME_LAST y NAME_LAST2, diseñado para culturas hispanoamericanas [15].

V-C. Microsoft Dynamics 365: DirPartyTable

Microsoft documenta oficialmente el patrón:

“A party is a person or an organization that is involved in a business. When you use the party concept, a person or an organization can play more than one role in a business (for example, customer, vendor, or contact).” [16]

Y justifica con un ejemplo concreto:

“A party can be both a customer and a vendor. [...] A party can be both an employee and a customer. For example, an employee of Contoso buys electronics from Contoso for personal use.” [16]

V-D. Oracle TCA: HZ_PARTIES

Oracle implementa la *Trading Community Architecture* (TCA), quizás la implementación más rigurosa del patrón. En TCA, un **Party** **no es** un cliente: se convierte en cliente cuando se crea un registro en HZ_CUST_ACCOUNTS. La misma entidad puede tener múltiples cuentas de cliente y ser simultáneamente proveedor. Es un modelo de tres capas: Party (identidad), Account (rol) y Site (ubicación) [17].

V-E. ERPNext: Party Type dinámico

ERPNext utiliza el campo polimórfico `party_type` + `party` mediante *Dynamic Links* del framework Frappe. Direcciones y contactos se comparten entre Customer, Supplier y Employee mediante este mecanismo [18].

V-F. Síntesis

La Tabla II resume el patrón. **Todos** los ERPs mayores del mundo usan una abstracción unificada de Persona/Entidad. Esto no es una coincidencia ni sobre-ingeniería: es la solución probada al problema real de que una misma entidad del mundo real cumple múltiples roles en un sistema de negocio.

Cuadro II
PATRÓN DE PERSONA UNIFICADA EN ERPS

ERP	Entidad central	Roles
Odoo	res.partner	Ranks
SAP	BUT000	BUT100
Dynamics	DirPartyTable	Party Roles
Oracle	HZ_PARTIES	Accounts
ERPNext	party_type	Dyn. Link

VI. SPLE: EL MARCO TEÓRICO QUE APLICA

El paradigma de *Software Product Line Engineering* (SPLE), estandarizado por el Software Engineering Institute (SEI) de Carnegie Mellon [11], define cómo construir un portafolio de productos a partir de un núcleo común. La tesis central es que cuando una organización **sabe** que construirá múltiples productos similares, la inversión anticipada en arquitectura compartida genera retornos económicos significativos.

La consultora opera exactamente en este escenario: múltiples proyectos, patrones comunes conocidos, y un objetivo estratégico explícito de reutilización. Esto contradice directamente la aplicación ingenua de YAGNI en contextos multi-proyecto, porque la reutilización es **conocida y planificada**, no especulativa.

VII. ANÁLISIS DE LA ANALOGÍA TALLER VS. FÁBRICA

La contrapropuesta argumenta que la analogía de taller versus fábrica es inadecuada porque “no fabricamos un único modelo de auto” [1]. La analogía se extiende a fabricar chasis, ejes y llantas, concluyendo que la fábrica “acabaría fabricando partes individuales pequeñas”, y que esto sería un desatino.

Sin embargo, eso es **exactamente** lo que hace la industria automotriz real. En este contexto, una *plataforma* es el conjunto de componentes base que comparten varios modelos de auto: la estructura del piso, las suspensiones, la ubicación del motor, y los sistemas eléctricos fundamentales. No es un auto completo: es la base común sobre la que se construyen autos muy diferentes.

La Tabla III muestra ejemplos concretos. Un Volkswagen Polo (hatchback económico) y un Audi Q3 (SUV premium) comparten la plataforma MQB. Un Chevrolet Onix (sedán), un Tracker (SUV) y una Montana (pickup) comparten la plataforma GEM. Un Toyota Corolla y un RAV4 comparten TNGA. Un Hyundai Tucson y un Kia Sportage — de marcas distintas— comparten la plataforma N3. Un Renault Duster y un Nissan Kicks comparten CMF-B [19]–[23].

Nadie diría que un Toyota Corolla y un RAV4 son “el mismo auto”, ni que un Hyundai Tucson y un Kia Sportage son “el mismo vehículo”. Son productos muy diferentes, para clientes muy diferentes, a precios muy diferentes. Pero comparten una base común que les permite a las automotrices no reinventar lo fundamental en cada modelo nuevo.

La analogía con la consultora es directa: un sistema para un colegio y un sistema para un hotel son productos diferentes, pero ambos necesitan autenticación, gestión de personas y auditoría. Esos componentes son la “plataforma” de la consultora. No es un sistema completo para todos: es la base probada sobre la que cada proyecto construye su funcionalidad específica.

Plataforma	Modelos en LATAM	Ahorro
VW MQB	Polo, T-Cross, Tiguan, Taos, Audi A3/Q3	30 %
GM GEM	Onix, Tracker, Montana	–
Toyota TNGA	Corolla, Corolla Cross, RAV4, Camry	20 %
Hyundai N3/K2	Tucson, Sportage, Creta, Seltos	–
CMF-B (RNM)	Duster, Kicks, Captur	30–40 %

VIII. SOBRE LA RESTRICCIÓN DE USO DE IA

La contrapropuesta establece que “la idea original, el flujo de trabajo, las entidades, sus nombres, propiedades y operaciones no pueden venir de una IA” [1]. Si bien la intención de evitar abstracciones especulativas es razonable, el criterio es arbitrario.

La calidad de una abstracción se mide por su resultado (cohesión, acoplamiento, expresividad del dominio), no por su origen. Un humano puede producir abstracciones especulativas tan deficientes como una IA, y una IA con el contexto adecuado puede identificar patrones que un humano pasa por alto. El criterio correcto es **revisión y validación por el equipo**, independientemente de quién o qué generó la propuesta inicial.

IX. SOBRE LAS LIMITACIONES RECONOCIDAS

La contrapropuesta reconoce honestamente sus propias limitaciones [1]: normalización con composición, dependencias opcionales entre plantillas, infraestructura (tablas propias vs. datos embebidos), manejo de errores, y aspectos de DDD como Aggregates, Bounded Contexts y Eventos.

Es notable que estos son **exactamente los problemas que un repositorio base institucional resuelve** al definir convenciones claras desde el inicio. La contrapropuesta identifica vacíos en su propia propuesta que la propuesta original ya aborda.

X. CONCLUSIONES

1. Se reconocen las críticas legítimas al primer informe: el diseño insuficiente de Persona, la falta de detalle operativo, y la necesidad de mayor consenso.
2. Se demuestra, con citas textuales de los autores originales (Fowler, Jeffries, Beck, Martin), que YAGNI no aplica a decisiones arquitectónicas ni a esfuerzos de mantenibilidad. Usar YAGNI para rechazar inversión arquitectónica es una malinterpretación del principio.
3. Se evidencia que la abstracción unificada de Persona/Contacto es el estándar de facto en la industria, adoptado independientemente por Odoo, SAP, Microsoft, Oracle y ERPNext.
4. Se demuestra que ambas propuestas convergen en el mismo mecanismo de distribución (copiar, adaptar, ser dueño del código), y que la diferencia es terminológica.
5. La industria automotriz valida el modelo de plataforma: Toyota, Volkswagen, GM, Hyundai-Kia y Renault-Nissan

construyen vehículos radicalmente distintos sobre bases compartidas, reduciendo costos entre un 20 % y 40 %.

6. Se propone que la discusión futura se enfoque en los detalles de implementación (gobernanza, contenido de las plantillas, proceso de revisión) en lugar de perpetuar una falsa dicotomía entre enfoques que son, en esencia, el mismo.

REFERENCIAS

- [1] F. E. Araoz Morales, “Reutilización basada en plantillas y composición como alternativa a librerías y OOP para la reutilización de código,” Área de Digitales – ACIDE, Arequipa, Perú, 2026.
- [2] G. E. Gutiérrez Castillo, “Reporte Técnico y Estratégico: Estándar Arquitectónico para la Reutilización de Software en una Consultora,” Área de Digitales – ACIDE, Arequipa, Perú, 2026.
- [3] M. Fowler, “Yagni,” *martinfowler.com*, 2015. [Online]. Available: <https://martinfowler.com/bliki/Yagni.html>. [Accessed: 16-Mar-2026].
- [4] R. Jeffries, “YAGNI, Skimping, and Technical Debt,” *ronjeffries.com*, 2019. [Online]. Available: <https://ronjeffries.com/articles/019-01ff/iter-yagni-skimp/>. [Accessed: 16-Mar-2026].
- [5] R. Jeffries, “You’re NOT Gonna Need It!,” *ronjeffries.com*. [Online]. Available: <https://ronjeffries.com/xprog/articles/practices/pracnotneed/>. [Accessed: 16-Mar-2026].
- [6] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Pearson, 2017.
- [7] M. Fowler, “Design Stamina Hypothesis,” *martinfowler.com*. [Online]. Available: <https://martinfowler.com/bliki/DesignStaminaHypothesis.html>. [Accessed: 16-Mar-2026].
- [8] M. Fowler, “Is High Quality Software Worth the Cost?,” *martinfowler.com*. [Online]. Available: <https://martinfowler.com/articles/is-quality-worth-cost.html>. [Accessed: 16-Mar-2026].
- [9] M. Fowler, “BeckDesignRules,” *martinfowler.com*. [Online]. Available: <https://martinfowler.com/bliki/BeckDesignRules.html>. [Accessed: 16-Mar-2026].
- [10] G. Fairbanks, *Just Enough Software Architecture: A Risk-Driven Approach*. Marshall & Brainerd, 2010.
- [11] P. Clements and L. Northrop, *Software Product Lines: Practices and Patterns*. Addison-Wesley, 2001.
- [12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [13] Odoo S.A., “res.partner source code,” *GitHub: odoo/odoo*, branch 17.0. [Online]. Available: https://github.com/odoo/odoo/blob/17.0/odoo/addons/base/models/res_partner.py. [Accessed: 16-Mar-2026].
- [14] SAP SE, “Business Partner in SAP S/4HANA,” *SAP Help Portal*. [Online]. Available: https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE. [Accessed: 16-Mar-2026].
- [15] “SAP BUT000 Table – Business Partner: General Data,” *leanx.eu*. [Online]. Available: <https://leanx.eu/en/sap/table/but000.html>. [Accessed: 16-Mar-2026].
- [16] Microsoft, “Party and global address book,” *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/data-entities/dual-write/party-gab>. [Accessed: 16-Mar-2026].
- [17] Oracle, “Trading Community Architecture (TCA),” *Oracle Documentation*. [Online]. Available: <https://docs.oracle.com/en/applications/>. [Accessed: 16-Mar-2026].
- [18] Frappe Technologies, “ERPNext party.py source code,” *GitHub: frappe/erpnext*. [Online]. Available: <https://github.com/frappe/erpnext/blob/develop/erpnext/accounts/party.py>. [Accessed: 16-Mar-2026].
- [19] Volkswagen AG, “Modular toolkit strategy as recipe for success: the MQB celebrates tenth anniversary,” *Volkswagen Newsroom*, 2022. [Online]. Available: <https://www.volkswagen-newsroom.com/en/press-releases/modular-toolkit-strategy-as-recipe-for-success-the-mqb-celebrates-tenth-anniversary-8>. [Accessed: 16-Mar-2026].
- [20] “GM GEM Vehicle Platform,” *GM Authority*. [Online]. Available: <https://gmauthority.com/blog/gm/gm-platforms/gem/>. [Accessed: 16-Mar-2026].
- [21] Toyota Motor Corporation, “Toyota New Global Architecture,” *global.toyota*. [Online]. Available: <https://global.toyota/en/mobility/tnga/index.html>. [Accessed: 16-Mar-2026].

- [22] “Hyundai-Kia N platform,” *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Hyundai-Kia_N_platforms. [Accessed: 16-Mar-2026].
- [23] Renault-Nissan-Mitsubishi Alliance, “Common Module Family (CMF),” 2013. [Online]. Available: https://en.wikipedia.org/wiki/Renault%E2%80%93Nissan_Common_Module_Family. [Accessed: 16-Mar-2026].
- [24] M. Lehtinen, “4 Most Important Software Development Principles: DRY, YAGNI, KISS, and SINE,” *mattilehtinen.com*. [Online]. Available: <https://mattilehtinen.com/articles/4-most-important-software-development-principles-dry-yagni-kiss-and-sine/>. [Accessed: 16-Mar-2026].